Reviewer Pack — Minimal Rank of Grothendieck Groups over Tensor Network Valu…

Entry ba465c024d35 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 11:47:28 UTC

Minimal Rank of Grothendieck Groups over Tensor Network Valuations

Entry ID: `ba465c024d35` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-26 11:47:28 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Grothendieck Groups) **Field B** (complexity object): Communication Complexity (Tensor Network)

Statement:

> For every n-variable CNF formula F, the minimal rank of its corresponding tensor network valuation T(F) is bounded by a function g(n) such that g(n) = O(2^n log n).

Rationale (proposer's reasoning):

> The minimal rank of Grothendieck groups provides a measure of complexity in algebraic geometry. Tensor network valuations are a representation of complexity in communication complexity. A relationship between these two fields could reveal inherent structures that lead to new complexity lower bounds or separations.

Taxonomy category: `BP_READTWICE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `318c608874f32e04`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the average minimal rank of tensor network valuations, calculated over 30 seeds, does not exceed O(2^n log n) for any n ≤ 40.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Grothendieck Groups" AND "tensor network valuations" - "minimal rank" AND ("CNF formula" OR "communication complexity") - "tensor network valuation" AND CNF AND Grothendieck GROUPS AND Big O(2^n log n)

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=-9, elapsed=179.7s

5.1 Generated Python source

```python
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2**n - 1):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            if all(clause[i] != -clause[j] for j in range(i)):
                clauses.append(clause)
        return clauses

    def tensor_network_valuation(cnf):
        n = len(cnf[0])
        T = [[0] * (2**n) for _ in range(2**n)]
        for clause in cnf:
            for i in range(2**n):
                if all((i >> j & 1) == abs(clause[j]) for j in range(n)):
                    T[i][i ^ sum([1 << j if clause[j] > 0 else 0 for j in range(n)] if clause[j] !
        return T

    def min_rank(matrix):
        m, n = len(matrix), len(matrix[0])
        rank = 0
        for i in range(min(m, n)):
            pivot_row = -1
            for r in range(i, m):
                if matrix[r][i] != 0:
                    pivot_row = r
                    break
            if pivot_row == -1:
                continue
            rank += 1
```

```
            for j in range(n):
                matrix[pivot_row][j] /= matrix[pivot_row][i]
            for r in range(m):
                if r != pivot_row and matrix[r][i] != 0:
                    for j in range(n):
                        matrix[r][j] -= matrix[pivot_row][j] * matrix[r][i]
        return rank

    def g(n):
        max_rank = 0
        for _ in range(30):  # Ensure at least 30 instances per seed
            cnf = generate_cnf(n)
            T = tensor_network_valuation(cnf)
            rank = min_rank(T)
            if rank > max_rank:
                max_rank = rank
        return max_rank

    n_values = [5, 10, 15, 20, 30, 40]
    results = []
    for n in n_values:
        results.append(g(n))

    avg_g_n = sum(results) / len(results)
    conjecture_holds = all(r <= 2**n * math.log(n) for r, n in zip(results, n_values))
    counterexample = "" if conjecture_holds else f"max_rank={max(results)} > O(2^n log n)"

    return {
        "metric_name": "g(n)",
        "metric_value": avg_g_n,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result["metric_value"])

    avg_metric_value = sum(results) / len(results)
    support_fraction = sum(r <= 2**n * math.log(n) for r, n in zip(results, [5, 10, 15, 20, 30, 40

    if all(r <= 2**n * math.log(n) for r, n in zip(results, [5, 10, 15, 20, 30, 40])):
        print(f"RESULT: SUPPORTED mean={avg_metric_value} std=NA support_fraction={support_fractio
    elif any(r > 2**n * math.log(n) for r, n in zip(results, [5, 10, 15, 20, 30, 40])):
        first_failing_seed = seeds[results.index(max(results))]
        print(f"RESULT: FALSIFIED counterexample=max_rank={max(results)} first_failing_seed={first
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
(empty)
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed before producing data, which means we cannot verify whether the average minimal rank of tensor network valuations meets the support condition. | next: Re-run the test to ensure it completes successfully and provides the necessary data for verification.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12905
2	preregistration	ollama_remote	glm4:latest	0	0	5625
3	novelty	ollama_remote	glm4:latest	0	0	4773
4	novelty	ollama_remote	glm4:latest	0	0	5487
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19767
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7240
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24760
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15446
9	judge	ollama_remote	glm4:latest	0	0	64282

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 160285 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/ba465c024d35.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ba465c024d35.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/ba465c024d35.tar.gz` (if generated)